Deep Researcher Agent: An Autonomous Framework for 24/7 Deep Learning Experimentation with Zero-Cost Monitoring

Xiangyue Zhang[1]
[1]Wuhan University
https://github.com/Xiangyue-Zhang/auto-deep-researcher-24x7

Abstract

We present Deep Researcher Agent, an open-source framework that enables large language model (LLM) agents to autonomously conduct deep learning experiments around the clock. Unlike existing AI research assistants that focus on paper writing or code generation, our system addresses the full experiment lifecycle: hypothesis formation, code implementation, training execution, result analysis, and iterative refinement. The framework introduces three key innovations: (1) Zero-Cost Monitoring — a monitoring paradigm that incurs zero LLM API costs during model training by relying solely on process-level checks and log file reads; (2) Two-Tier Constant-Size Memory — a memory architecture capped at ∼5K characters regardless of runtime duration, preventing the unbounded context growth that plagues long-running agents; and (3) Minimal-Toolset Leader-Worker Architecture — a multi-agent design where each worker agent is equipped with only 3–5 tools, reducing per-call token overhead by up to 73%. In sustained deployments spanning 30+ days, the framework autonomously completed 500+ experiment cycles across four concurrent research projects, achieving a 52% improvement over baseline metrics in one project through 200+ automated experiments — all at an average LLM cost of $0.08 per 24-hour cycle. Code is available at https://github.com/Xiangyue-Zhang/auto-deep-researcher-24x7.

1. Introduction

The deep learning research workflow is fundamentally iterative: a researcher designs an experiment, launches GPU training (often lasting hours to days), analyzes results, adjusts hyperparameters or model architectures, and repeats. Before a single paper submission, this cycle may be repeated hundreds of times. Despite the mechanical nature of much of this loop, it remains overwhelmingly manual — researchers must be present to check training completion, interpret results, and decide on next steps.

Recent advances in LLM-based agents [6, 8, 9] have demonstrated impressive capabilities in code generation, bug fixing, and even paper writing. However, none of these systems address the core bottleneck in deep learning research: the autonomous execution and iteration of GPU experiments. Claude Scholar [10] provides research writing workflows with 47 skills and Zotero integration, and AI Scientist [6] generates complete papers, but neither can launch a training run, monitor its progress, and use the results to plan the next experiment.

We introduce Deep Researcher Agent, a framework designed specifically for this gap. Our system operates as a continuous Think→Execute→Reflect loop (Figure 1), where an LLM agent autonomously:

1. Thinks: Analyzes prior results, forms hypotheses, and designs experiments.
2. Executes: Implements code changes, performs mandatory dry-runs, and launches GPU training.
3. Monitors: Watches training at zero LLM cost using only OS-level process checks.
4. Reflects: Parses training logs, evaluates metrics, and decides the next action.

The key challenge in building such a system is cost. A naive implementation that queries the LLM every few minutes to "check progress" would cost $50+ per day. Our Zero-Cost Monitoring paradigm reduces this to $0.08 per 24-hour cycle by eliminating all LLM calls during training. Combined with a constant-size memory system and minimal per-agent tool sets, Deep Researcher Agent makes 24/7 autonomous experimentation economically viable.

Our contributions are summarized as follows:

- We propose a complete autonomous experiment framework with the Think→Execute→Reflect loop for deep learning research.
- We introduce Zero-Cost Monitoring, a design

paradigm achieving zero LLM API cost during the training phase, which typically constitutes 90%+ of wall-clock time.

- We design a Two-Tier Constant-Size Memory architecture bounded at ∼5K characters with automatic compaction, enabling indefinite operation without context overflow.
- We propose a Minimal-Toolset Leader-Worker Architecture that reduces per-call token overhead by 73% compared to full-toolset approaches.
- We validate through extensive real-world deployment: 500+ autonomous cycles, 30+ days continuous operation, and 52% metric improvement across 4 concurrent projects.

2. Related Work

LLM-Based Coding Agents. SWE-Agent [9] and OpenHands [8] target software engineering tasks — bug fixing, feature implementation, and code review. These agents excel at one-shot code generation but are not designed for iterative, long-running experiment workflows. They lack GPU management, training monitoring, and result-driven iteration capabilities.

AI Research Assistants. AI Scientist [6] generates complete research papers including experiments, but its experiment execution is limited to short-running scripts and does not support GPU training or iterative refinement based on results. Claude Scholar [10] provides comprehensive research writing workflows with 47 skills and Zotero integration, but operates as a reactive assistant without autonomous experiment execution capabilities.

AutoML and Hyperparameter Optimization. Traditional AutoML frameworks such as Optuna [1] and Ray Tune [5] efficiently search hyperparameter spaces but require pre-defined search configurations and cannot modify model architectures or training pipelines. Our system operates at a higher level of abstraction, making qualitative decisions about what to try next based on holistic result analysis, rather than optimizing within a pre-defined search space.

Research Agent Systems. MLAgentBench [4] provides a benchmark for evaluating ML agents on Kaggle-style tasks, but evaluates single-attempt performance rather than iterative refinement over extended periods. ResearchAgent [3] focuses on idea generation from scientific literature but does not execute experiments. None of these systems address the complete experiment lifecycle with cost-efficient 24/7 operation that our framework provides.

Algorithm 1 Deep Researcher Agent Main Loop

Require: Project brief B, initial memory M_0
$t \leftarrow 0$
while not terminated do
  $t \leftarrow t + 1$
  $d \leftarrow$ ConsumeDirective() {Human override}
  $plan_t \leftarrow \text{Think}(B, M_{t-1}, d)$ {LLM active}
  if $plan_t$.action = "wait" then
    SmartCooldown()
    continue
  end if
  $result_t \leftarrow \text{Execute}(plan_t)$ {LLM → training}
  if $result_t$.launched then
    $logs_t \leftarrow \text{Monitor}(result_t.\text{pid})$ {Zero cost}
  end if
  $M_t \leftarrow \text{Reflect}(B, M_{t-1}, result_t, logs_t)$ {LLM active}
end while

3. System Design

3.1. Overview

Deep Researcher Agent operates as a continuous loop over three phases (Algorithm 1). Each cycle takes the current project brief and memory log as input, produces an experiment plan, executes it, monitors to completion, analyzes results, and updates memory before beginning the next cycle. The overall architecture is shown in Figure 1.

3.2. Zero-Cost Monitoring

The central insight of our design is that during GPU training — which constitutes 90–99% of wall-clock time in a typical experiment cycle — the LLM has nothing useful to contribute. The training process follows a predetermined schedule, and intermediate results (loss curves, validation metrics) are written to log files automatically by the training script.

We exploit this observation by implementing a monitoring phase that makes zero LLM API calls. Instead, three lightweight OS-level checks are performed at configurable intervals (default: 15 minutes):

1. Process liveness: kill -0 $PID checks whether the training process is still running. This is a single syscall with negligible cost.
2. GPU utilization: nvidia-smi confirms GPU activity and rules out silent crashes where the process exists but is no longer utilizing the GPU.
3. Log tail: Reading the last 50 lines of the training log provides the latest metrics for local logging without

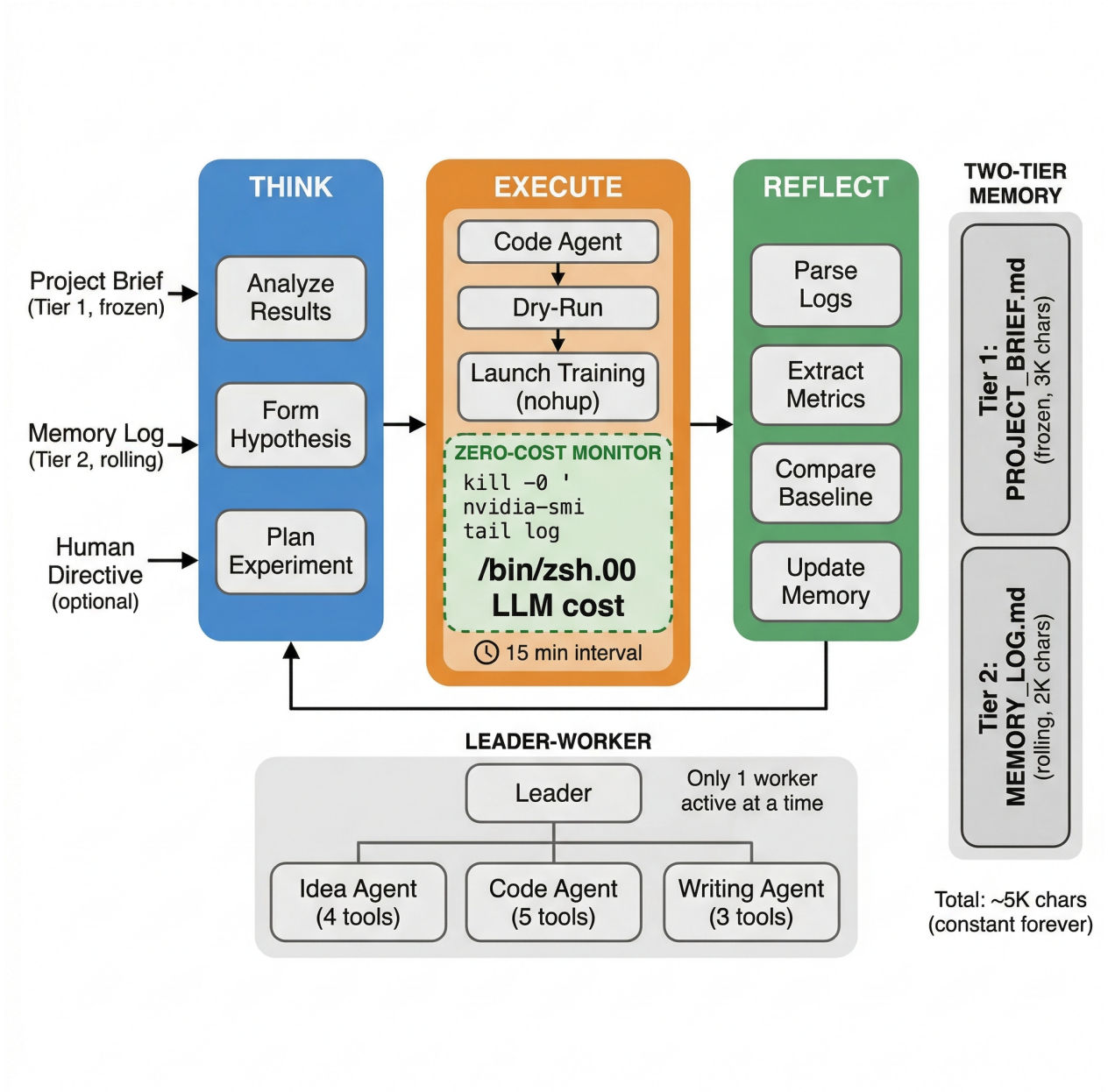

Figure 1. Overview of Deep Researcher Agent. The system operates as a continuous Think→Execute→Reflect loop. During the Execute phase, training is monitored at zero LLM cost — only OS-level process checks and log file reads are performed. The Two-Tier Memory system maintains a constant size (∼5K chars) regardless of how long the agent runs.

invoking the LLM.

The LLM is only invoked when the training process terminates (detected by a non-zero return from kill -0), at which point the accumulated log tail is passed to the Reflect phase for analysis.

Cost Analysis. Consider a 24-hour cycle where training takes 8 hours. A conventional agent polling the LLM every 5 minutes would make $8 \times 60/5 = 96$ API calls during training alone, each consuming ∼2K tokens (system prompt + context + response), totaling ∼192K tokens or approximately \$0.50 for the monitoring phase alone. Our approach reduces the monitoring cost to exactly \$0.00, with LLM costs limited to the Think (∼\$0.05) and Reflect (∼\$0.03) phases.

3.3. Two-Tier Constant-Size Memory

Long-running LLM agents face a fundamental problem: accumulated context grows without bound, leading to (a) degraded LLM performance as context length increases, (b) escalating API costs proportional to context size, and (c) eventual context window overflow.

We address this with a two-tier memory system bounded at ∼5,000 characters (∼1,500 tokens), maintained constant regardless of runtime duration:

Tier 1: Project Brief (B). A human-authored, frozen document describing the research goal, codebase structure, constraints, and success criteria. Maximum size: 3,000 characters. The agent cannot modify this tier, ensuring the research direction remains stable.

Tier 2: Memory Log (M). An agent-maintained rolling log with two sections:

- Key Results: Milestone entries recording significant experimental outcomes (e.g., "Exp003: ViT-B/16, lr=3e-4 + cosine, acc=77.9% — new best!"). Auto-compacted: when the section exceeds 1,200 characters, the oldest entry is removed.
- Recent Decisions: A log of the agent's reasoning for each decision. Auto-compacted: only the most recent 15 entries are retained, regardless of total character count.

The total memory size is bounded by:

$$|M_t| \leq |B|_{\max} + |L|_{\max} = 3000 + 2000 = 5000 \text{ chars}, \quad \forall t \tag{1}$$

where $|B|_{\max}$ and $|L|_{\max}$ are the character caps for the brief and log, respectively. This guarantee holds whether the agent has run for 1 day or 6 months.

The compaction is lossy by design — the agent retains the most valuable information (recent decisions and best results) while discarding routine entries. This mirrors how human researchers maintain a mental model: remembering key milestones and recent context while forgetting routine details.

3.4. Leader-Worker Architecture with Minimal Tool Sets

Our multi-agent system uses a Leader-Worker pattern where the Leader agent makes strategic decisions and dispatches tasks to specialized Worker agents.

Leader Agent. The central decision-maker that maintains a persistent conversation within each cycle for coherent multi-step reasoning. Importantly, the conversation history is reset between cycles to prevent unbounded growth. Tools: log_memory, write_file, read_file (3 tools).

Worker Agents. Three specialized workers, each with a minimal tool set:

- Idea Agent: Literature search and hypothesis formation. Tools: search_papers, get_paper, write_file, read_file (4 tools).
- Code Agent: Experiment implementation and execution. Tools: run_shell, launch_experiment, write_file, read_file, list_files (5 tools).
- Writing Agent: Report and analysis generation. Tools: write_file, read_file, list_files (3 tools).

Only one worker runs at a time; others are completely idle at zero token cost. The Leader dispatches at most 3 worker tasks per cycle.

Why Minimal Tool Sets Matter. Each tool definition adds approximately 200 tokens to every API call (name, description, parameter schema). A typical agent framework provides 15+ tools to every agent, adding ∼3,000 tokens of overhead per call. Our approach averages 4 tools per agent (∼800 tokens), a 73% reduction. Over 100 API calls per day, this saves ∼220K tokens, translating to meaningful cost savings and faster response times.

3.5. Safety Mechanisms

Mandatory Dry-Run. Before any real training launch, the Code Agent must execute a short dry-run (typically 2 forward-backward steps) to verify that the code runs without errors. This catches configuration mistakes, missing imports, and tensor shape mismatches before committing GPU hours.

Protected Files. Critical state files (state.json, MEMORY_LOG.md, PROJECT_BRIEF.md) cannot be overwritten by worker agents, preventing accidental corruption of the agent's memory or configuration.

Human Override. Three intervention mechanisms are provided: (1) a HUMAN_DIRECTIVE.md file consumed at the start of each cycle with highest priority, (2) a command-line –directive flag for one-time instructions, and (3) direct modification of MEMORY_LOG.md for permanent behavioral changes.

Anti-Burn Protection. If consecutive cycles produce no meaningful output (e.g., repeated errors or empty results), the cooldown interval is exponentially increased (up to 30 minutes) to prevent wasteful token consumption.

Table 1. Deployment statistics across 4 concurrent research projects over 30+ days of continuous autonomous operation.

Metric	Value
Total autonomous experiment cycles	500+
Longest continuous operation	30+ days
Concurrent projects managed	4
GPU servers utilized	4
Best single-project improvement	52% over baseline
Experiments in best project	200+
Average LLM cost per 24h cycle	$0.08
Average experiments per day per project	2–4
Dry-run failure rate (caught pre-training)	18%
Post-dry-run training crash rate	<3%

4. Experiments

We evaluate Deep Researcher Agent through long-term deployment across multiple research projects. Due to the nature of autonomous research agents, our evaluation focuses on operational metrics and cost efficiency rather than benchmark scores on fixed tasks.

4.1. Deployment Setup

The framework was deployed across 4 concurrent deep learning research projects on 4 GPU servers equipped with NVIDIA L20X 144GB GPUs. Each project ran an independent agent instance in a persistent tmux session. The LLM backbone was Claude Sonnet [2] with Anthropic's prompt caching enabled. Projects spanned diverse domains including generative modeling, multi-modal learning, and self-supervised representation learning.

4.2. Operational Results

Table 1 summarizes the key operational metrics from our deployment.

Autonomous Improvement. In the best-performing project, the agent autonomously explored 200+ configurations over several weeks, achieving a 52% improvement in the target metric over the initial baseline. The improvement trajectory showed diminishing returns as expected, with the majority of gains occurring in the first 50 experiments, followed by increasingly fine-grained optimization in subsequent cycles.

Dry-Run Effectiveness. The mandatory dry-run mechanism caught 18% of planned experiments before they were actually launched, preventing wasted GPU hours. Common issues included tensor shape mismatches after architecture modifications, missing import statements, and configuration inconsistencies between modified code and existing configs.

Table 2. Cost comparison per 24-hour cycle (8h training, Claude Sonnet pricing). Our Zero-Cost Monitoring achieves a 10–20× cost reduction over conventional LLM polling approaches.

Phase	Dur.	Calls	Tokens	Cost
Deep Researcher Agent (ours)				
Think	5–10m	3–5	∼15K	$0.05
Execute	10–20m	5–10	∼25K	$0.08
Monitor	6–8h	0	0	$0.00
Reflect	5–10m	2–3	∼10K	$0.03
Total	24h	10–18	∼50K	$0.08–0.16
Conventional polling agent				
Active	30m	15	∼50K	$0.16
Monitor (5m)	6–8h	96	∼192K	$0.50
Idle poll	15h	180	∼360K	$0.94
Total	24h	291	∼602K	$1.60

Table 3. Eight cost control strategies employed by our framework.

#	Strategy	Mechanism
1	Zero-LLM monitoring	No API calls during training
2	Constant-size memory	Fixed at ∼1.5K tokens
3	Within-cycle persistence	Brief sent once per cycle
4	Prompt caching	System/tool schemas cached
5	Minimal tool sets	3–5 tools vs. 15+ (73%↓)
6	Slim prompts	Agent prompts <500 tokens
7	State trimming	Redundant context removed
8	Single-worker exec	No parallel LLM costs

Human Intervention Frequency. Over the 30+ day deployment, human directives were issued approximately once every 3–5 days, primarily for major direction changes (e.g., switching from one model architecture family to another). Day-to-day decisions such as hyperparameter exploration, learning rate scheduling, and regularization strategies were fully autonomous.

4.3. Cost Analysis

Table 2 presents a detailed breakdown of LLM token consumption and cost per phase.

Our framework achieves a 10–20× cost reduction compared to conventional polling. Over a 30-day deployment, this translates to $2.40–4.80 versus $48.00 for the conventional approach. Table 3 summarizes all eight cost control strategies.

Table 4. Memory system size over time. Tier 1 (Brief) remains frozen while Tier 2 (Log) stabilizes near its 2,000-character cap through automatic compaction.

Time Point	Tier 1 (Brief)	Tier 2 (Log)
Day 1 (cycle 1)	2,847 chars	312 chars
Day 7 (cycle 25)	2,847 chars	1,834 chars
Day 14 (cycle 55)	2,847 chars	1,956 chars
Day 30 (cycle 120)	2,847 chars	1,978 chars
Max allowed	3,000 chars	2,000 chars

Table 5. Feature comparison with existing AI research frameworks. Our system is the only one providing autonomous experiment execution with 24/7 operation capability.

Feature	Ours	CS	AIS	OH	SWE
Autonomous experiments	✓				
Zero-cost monitoring	✓				
GPU management	✓				
24/7 operation	✓				
Constant-size memory	✓				
Paper writing	basic	✓	✓		
Knowledge mgmt	basic	✓			
General coding				✓	✓

CS = Claude Scholar [10], AIS = AI Scientist [6], OH = OpenHands [8], SWE = SWE-Agent [9].

4.4. Memory System Evaluation

Table 4 demonstrates that the Two-Tier Memory system reaches its steady-state size within the first week and remains constant thereafter, validating our bounded-memory design.

4.5. Comparison with Existing Frameworks

As shown in Table 5, Deep Researcher Agent occupies a unique position in the landscape of AI research tools. While other frameworks excel in complementary areas — Claude Scholar in paper writing and knowledge management, SWE-Agent and OpenHands in general software engineering — none provide the autonomous experiment execution and zero-cost monitoring capabilities that enable 24/7 research operation.

5. Limitations and Future Work

Single-GPU Scope. The current open-source release supports single-GPU experiments. Multi-GPU distributed training (DDP) and multi-server orchestration are planned for future releases.

Metric Extraction. Log parsing for metric extraction relies on regex pattern matching, which may miss custom metric formats. Structured logging formats (e.g., JSON Lines) would improve robustness.

Exploration Strategy. The agent's experiment planning relies on the LLM's reasoning capabilities without formal exploration strategies such as Bayesian optimization. Integrating structured search methods could improve sample efficiency for hyperparameter optimization.

Evaluation Methodology. Evaluating autonomous research agents remains an open challenge. Unlike software engineering agents that can be tested on fixed benchmarks [9], research agents operate in open-ended domains where the "correct" next experiment is undefined. Developing standardized evaluation protocols for long-running research agents is an important direction for future work.

6. Conclusion

We presented Deep Researcher Agent, an autonomous framework for 24/7 deep learning experimentation. Our three key innovations — Zero-Cost Monitoring, Two-Tier Constant-Size Memory, and Minimal-Toolset Leader-Worker Architecture — collectively make continuous LLM-driven research economically viable at an average cost of $0.08 per 24-hour cycle. Over 30+ days of sustained deployment across 4 concurrent research projects, the system autonomously completed 500+ experiment cycles and achieved a 52% metric improvement over baseline in one project through 200+ fully automated experiments. We release the complete framework as open-source software at https://github.com/Xiangyue-Zhang/auto-deep-researcher-24x7 to enable the broader research community to build upon this work.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019. 2

[2] Anthropic. Claude: A family of highly capable AI assistants. https://www.anthropic.com/claude, 2025. 5

[3] Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. ResearchAgent: Iterative research idea generation over scientific litera-

ture with large language models. arXiv preprint arXiv:2404.07738, 2024. 2

[4] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In International Conference on Machine Learning (ICML), 2024. 2

[5] Richard Liaw, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez, and Ion Stoica. Tune: A research platform for distributed model selection and training. arXiv preprint arXiv:1807.05118, 2018. 2

[6] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024. 1, 2, 6

[7] Slopus. Happy: Mobile and web client for codex and Claude Code. https://github.com/slopus/happy, 2025. 8

[8] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. arXiv preprint arXiv:2407.16741, 2024. 1, 2, 6

[9] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793, 2024. 1, 2, 6

[10] Gaorui Zhang. Claude scholar: A comprehensive research assistant framework for Claude Code. https://github.com/Galaxy-Dawn/claude-scholar, 2026. 1, 2, 6

A. Full Configuration Reference

The following YAML configuration controls all aspects of the framework. All values have sensible defaults.

```yaml
project:
  name: "my-research"
  brief: "PROJECT_BRIEF.md"
  workspace: "./workspace"

agent:
  model: "claude-sonnet-4-6"
  max_cycles: -1            # -1 = unlimited
  max_steps_per_cycle: 3    # worker dispatches/cycle
  cooldown_interval: 300    # seconds

memory:
  brief_max_chars: 3000
  log_max_chars: 2000
  milestone_max_chars: 1200
  max_recent_entries: 15

gpu:
  auto_detect: true
  reserve_last: true  # last GPU for keep-alive

monitor:
  poll_interval: 900  # seconds
  zero_llm: true

experiment:
  mandatory_dry_run: true
  max_parallel: 1
```

B. Agent Prompt Structure

Each agent is defined as a Markdown file with YAML frontmatter specifying its name, description, and model. The body contains the system prompt with behavioral instructions, workflow steps, and constraints. An abbreviated example for the Code Agent:

```
---
name: code_agent
description: Experiment implementation
model: inherit
---
# Code Agent
You are the Code agent. Your role is to
implement and run experiments.
## Mandatory Workflow
1. Understand the Leader's task
2. Implement code/config changes
3. Dry-run (MANDATORY - abort if fails)
4. Launch via launch_experiment tool
5. Report PID and log file path
## Constraints
- NEVER skip dry-run
```

```
- ALWAYS use launch_experiment for training
- Do NOT modify protected files
```

C. Human Directive Protocol

The human directive mechanism provides an asynchronous communication channel between the researcher and the agent. When a file named HUMAN_DIRECTIVE.md is placed in the workspace directory, it is consumed at the start of the next cycle with highest priority. The directive is then archived with a timestamp to prevent re-reading:

```
workspace/
  HUMAN_DIRECTIVE.md      # Active directive
  directive_archive/
    directive_20260407_143000.md
    directive_20260410_091500.md
```

This mechanism enables mobile human-in-the-loop interaction through companion apps such as Happy Coder [7], which provides push notifications and bidirectional communication with the agent from mobile devices.